Hierarchical Planning for the Cuboid Multi-Agent Collective Construction (Cuboid-MACC) Problem

***Abstract*— The Multi-Agent Collective Construction (MACC) problem tasks teams of mobile robots with assembling structures much larger than themselves. However, existing MACC planners restrict these robots to using only uniform cubic $(1 \times 1 \times 1)$ blocks. In this work, we create a new planner that solves the MACC problem with *cuboid* $(n \times 1 \times 1)$ blocks, which we call the Cuboid-MACC problem. In doing so, we can now build structures with hollow features. The cuboid blocks introduce new challenges beyond those present in MACC: blocks are no longer rotationally symmetric, and a carried block can collide with other blocks or robots; both features restrict the paths robots can take while carrying blocks. To address Cuboid-MACC, we present a novel hierarchical planning approach that handles the maneuverability constraints imposed by cuboid blocks. First, we use A^* to determine a sequence of abstract actions (block placements and removals) to build the structure without planning specific paths for the robots. We nevertheless ensure that such paths will exist by requiring each abstract action to be executable, i.e., to have reachable access poses before and after the action. Next, we identify dependencies between the chosen abstract actions by checking for conflicts between single-agent paths to execute the actions and store the dependencies as an acyclic Action Dependency Graph. Finally, we iterate on the specific paths taken by robots using a low-level multi-agent pathfinding algorithm by suitable modifications to conflict-based search (CBS). We demonstrate our planner on 200 randomly generated structures built with three types of cuboid blocks.**

I. Introduction

While humans continue to play a crucial role in many construction projects, recent years have seen an increased interest in studying autonomous robotic systems for building structures in remote and hostile situations on Earth, in space, or on other planets [1]–[8]. When structures are much larger than the robots that will build them, planning algorithms must ensure not just the completion of the structure, but that at every phase of construction it is possible for the robots to traverse the structure to accomplish their tasks. Inspired by termites that can build mounds that are significantly larger than themselves, prior works study a collective construction problem where multiple robots are tasked to manipulate building "blocks" to assemble a much larger structure. This is referred to as the Multi-Agent Collective Construction (MACC) problem [9].

Most previous works on MACC [10]–[14] have focused on constructing three-dimensional grid structures using identical $(1 \times 1 \times 1)$ cube blocks. While this abstraction simplifies planning and coordination, it significantly restricts the geometric expressiveness of the target structure. As a step towards practical construction scenarios, we generalize the building blocks from uniform unit cubes to heterogeneous cuboids (elongated cuboid blocks with different sizes) $(n \times 1 \times 1)$, which we call the **Cuboid Multi-Agent Collective Construction (Cuboid-MACC)** problem.

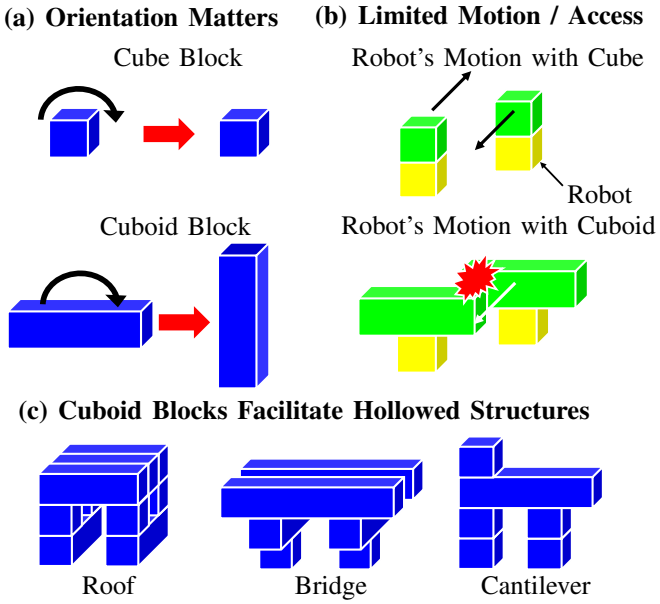


Fig. 1: Key differences between Cube-MACC and Cuboid-MACC. (a) Orientation matters for cuboid blocks but not for unit cubes, introducing an additional planning dimension. (b) Cuboids impose stricter motion and collision constraints on robots during transport. (c) Elongated cuboids enable hollowed geometries such as roofs, bridges, or cantilevers.

Compared to previous MACC (i.e., Cube-MACC) formulations, cuboid blocks introduce significantly greater combinatorial and geometric complexity, arising from:

1) An additional planning dimension (orientation) introduced by cuboid blocks (Fig. 1(a));
2) More restrictive collision-avoidance constraints due to their larger footprint (Fig. 1(b)); and
3) The strict scaffolding requirements needed to safely lift cuboid blocks to construct overhanging features (Fig. 1(c)) without collisions.

To the best of our knowledge, neither this modified Cuboid-MACC formulation nor a planner capable of addressing its unique maneuverability constraints has been presented in prior work. However, for the Cube-MACC problem, prior work [14] has shown that hierarchical planning achieves higher-quality solutions than heuristic [10], [15], [16] or learning-based [11], [17] approaches while incurring substantially lower computational cost than optimization-

based formulations [12]. We therefore adopt a hierarchical framework and extend it to the more challenging Cuboid-MACC setting, where block orientation and carried-block collision constraints substantially tighten feasibility during transport and placement.

To address these challenges, we propose a three-level hierarchical planner that remains tractable while producing feasible multi-robot plans. As shown in Fig. 2, the proposed architecture comprises:

- **High-level planning**: An A* search computes a valid sequence of block placements and removals to build the target structure under a single-agent assumption.
- **Interface layer**: These paths are analyzed for spatial and temporal conflicts to extract dependencies, which are encoded into an acyclic Action Dependency Graph.
- **Low-level planning**: A customized task allocation method [18] combined with a modified Conflict-Based Search (CBS) [19] computes the final, collision-free multi-robot paths.

The core contributions of this work are:

1) Formalizing the Cuboid-MACC problem to support heterogeneous blocks and hollow structure assembly.
2) A scalable, hierarchical planner generating feasible, collision-free multi-robot construction sequences.
3) Adapting CBS to manage dynamic environments by resolving three novel multi-agent conflicts.
4) Empirical validation of the planner on 200 randomly generated structures requiring complex coordination.

Section II reviews related work, followed by the Cuboid-MACC problem formulation in Section III. Section IV details our hierarchical planner, with experimental results and conclusions presented in Sections V and VI. To support reproducibility, we will release the source code, the test-structure generator, and the full evaluation set upon publication.

II. Related Works

Hierarchical Planning methods leverage problem structure and domain knowledge to design hierarchies of planners that each solve an easier problem. Plans can be generated using hand-designed abstractions, or high-level actions, then refined into appropriate low-level actions. Recent work in multi-arm assembly [20] generates roadmaps, then solves a relaxed assembly problem on these roadmaps, before finding arm trajectories. Another work on self-assembling structures [15] calculates a partial order plan, then derives motion plans using local controllers. Additionally, methods propose ways to optimize refining abstract plans [21] to further reduce computational expense. In this work, we employ a hierarchical planner to solve a modified MACC problem.

Multi-agent Collective Construction: The TERMES [9] project develops the problem of collective construction of three-dimensional structures using small mobile robots. The robots can pick-up and deliver blocks that compose the structure, and climb stairs of blocks to traverse the structure. [10] develops a heuristic planner to solve this problem. They employ a Minimum Spanning Tree-based search method,

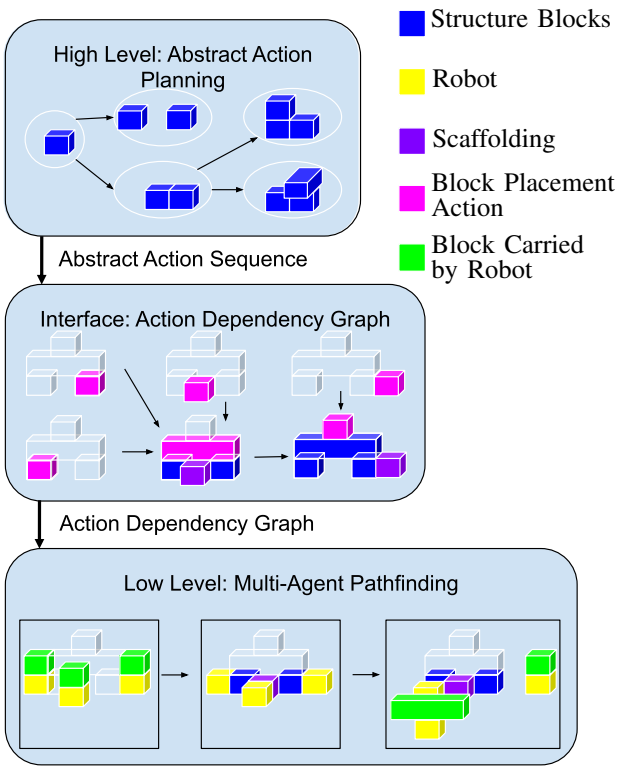


Fig. 2: The high-level planner finds a sequence of block placements and removals that builds the target structure. The interface identifies dependencies between these actions. The low-level finds multi-agent paths to accomplish the actions from the high-level.

which plans quickly even for large structures. Another method [11] uses Distributed Reinforcement Learning for the collaboration of multiple agents without explicit interactions. Although fast to plan, these methods require very large numbers of robot actions. An alternative "exact" method uses Mixed Integer Linear Programming (MILP) to find a plan optimal in the number of timesteps (or makespan) [12]. Decomposing the goal structure into multiple smaller structures can reduce computation time in this method [13]. Decomposition is effective in reducing the computational effort for sparse and scattered structures compared to the exact method, but less so for tall or densely built structures. Another work [14] uses a hierarchical planning framework for the MACC problem that plans for robot construction tasks and multi-robot paths hierarchically.

In this paper, we modify the MACC problem to include cuboid-shaped blocks. Robot path planning is more constrained than in Cube-MACC because the carried block's yaw affects which motions are collision-free. We build on existing hierarchical methods for MACC [14], as well other domains [15], [20], [21] to develop a hierarchical planning pipeline (Fig. 2) for our modified problem. After planning an assembly sequence (Abstract Action Planning) that respects structure stability constraints, we identify dependencies between assembly actions (Action Dependency Graph), then solve a multi-agent pathfinding problem for robots carrying heterogeneous blocks.

Multi-agent Pathfinding (MAPF) problems find collision-free paths for a group of agents to reach target locations while optimizing an objective. The work in [22] formalizes a classical MAPF problem as a tuple (G, s, t) for k agents, where $G = (V, E)$ is an undirected graph, and s:$[1, ..., k] \rightarrow V$ and t:$[1, ..., k] \rightarrow V$ map agents to source and target vertices respectively. Time is discrete and each agent can take a single action at every time step. We implement and modify [19] to find shortest paths for multiple agents transporting and delivering blocks. A variant of MAPF, the Multi-agent pick-up and delivery (MAPD) problem [23], extends MAPF solutions to include task planning and task allocation to multiple agents. Although there has been significant development of solutions for large-scale MAPF and MAPD problem, path planning for robots with rigid polygonal or polyhedral shapes is understudied in multi-agent settings.

Path planning for polygonal robots often employs complex collision-checking computations on robot configuration spaces [24]–[26]. In a multi-robot scenario, the cost for collision-checking calculations can grow as the square of the number of robots. Using these algorithms thus leads to an explosive number of calculations if precise collision avoidance is required. Our work mitigates this by using conservative collision envelopes. Robots are modeled as occupying exactly one grid cell. During rotation, both the robot cell and all cells occupied by the carried block must remain collision-free. Thus, a rotation is permitted only if the entire swept envelope of the robot-block system is empty.

III. Cuboid Multi-Agent Collective Construction Problem

For the Cuboid-MACC problem, we first formalize the static world model and support constraints, then define robot states and primitive actions, and finally introduce validity conditions required for planning. As this is the first paper to address the cuboid-MACC problem, we give a number of formal definitions throughout the text to rigorously define the problem class and introduce language useful for discussing its challenges. It is worth noting that we rely on *cubic* ($1\times1\times1$) mobile robots to construct a specified structure composed of both cubic ($n = 1$) and cuboid ($n \times 1 \times 1$) blocks.

We use a right-handed coordinate system; the z-axis is vertical and the x-y plane is the ground. A block's geometry is specified by its length n; in this work, all blocks are 1 unit tall (along z) and 1 unit deep in the horizontal plane. We restrict attention to odd block lengths $n \in \{1, 3, 5\}$, so the block center aligns with a grid cell center. A block or robot pose is denoted by $L = (x, y, z, r)$, where (x, y, z) specifies the coordinates of its center and r denotes its rotation about the z-axis (yaw). Because the robots and blocks are symmetric under a 180° yaw about their center, orientations 0° and 180° are equivalent (and likewise 90° and 270°). We therefore restrict $r \in \{0°, 90°\}$.

The ground level corresponds to $z = 0$. A special depot is available outside the grid world, where an infinite supply of blocks can be accessed. Robots can carry blocks; when carried by a robot, the block's center is positioned 1 unit above the robot, and its rotation matches that of the robot.

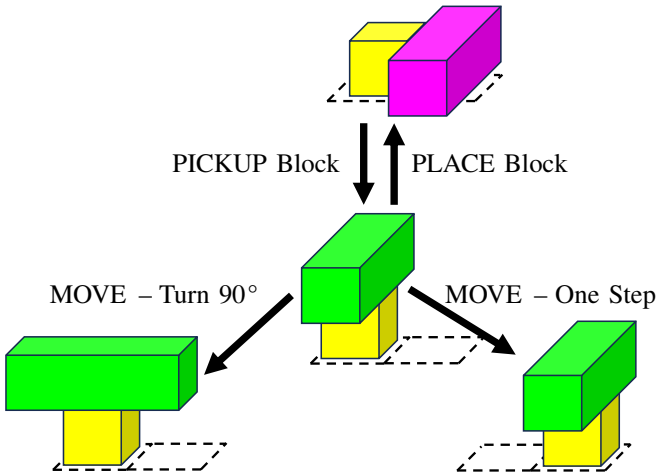


Fig. 3: An illustration of the primitive actions where a robot (Yellow) carrying an elongated block (Green) can: WAIT; MOVE one step forward or backward; MOVE to rotate by 90°; and PLACE a block (Pink) forward or backward. Carried blocks extend to the left and right of the robot.

Definition 1. *A **world state** $\{(B_1, L_1)...(B_N, L_N)\}$ is a set of pairs of blocks B_i and their poses L_i in the world. For every cell in the grid world, a world state uniquely determines if that cell is filled or not.*

Definition 2. *Let $F(B, L)$ denote the set of grid cells occupied by block B when placed at pose $L = (x, y, z, r)$. Then B has support at L if:*

- *$z = 0$, or*
- *the cell directly below $(x, y, z - 1)$ is filled, or*
- *there exist two filled cells in $\{(x', y', z-1) \mid (x', y', z) \in F(B, L)\}$ that are symmetric about (x, y) along axis r.*

Definition 3. *A **robot state** $X = (t, L, B)$ specifies a timestep t, robot pose L, and the carried block B.*

Definition 4. *For a block pose (x_b, y_b, z_b, r_b), the **access poses** are $\{(x_b - 1, y_b, z_b, r_b), (x_b + 1, y_b, z_b, r_b)\}$ if r_b is 0, and $\{(x_b, y_b - 1, z_b, r_b), (x_b, y_b + 1, z_b, r_b)\}$ if r_b is 90.*

In this work, we distinguish between abstract actions and primitive actions, similar to [14]. There are two types of abstract action: ADD, which inserts a block into the world state at a pose with support, and REMOVE, which deletes a block from a specific pose in the world state.

There are four types of primitive actions, each requiring one time step: WAIT, MOVE, PLACE, and PICKUP. The actions available to a robot carrying a block of size ($3\times1\times1$) are shown in Fig. 3. WAIT is available only if the robot is at the depot. Robots may MOVE up to 1 unit forwards or backwards or rotate 90 degrees in place. As part of a MOVE, the robot can change its height by up to one unit as if on a staircase. The robot may MOVE between ground level in any cell at the edge of the world and the depot. In-place yaw

rotations are a MOVE whose start and end poses differ only in r. Blocks can be PLACEd in front of or behind the robot, at the same height as the robot. The resulting block pose has the same height and rotation as the robot; x or y will differ by 1 unit. A block can only be placed at a pose that has support. PICKUP is available if the robot is at an access location of a block. Note that in the Cube-MACC problem, size 1 blocks can be picked up or placed from any direction, which is not permitted in this formulation. WAIT has 0 cost; other actions cost 1 unit. As robots can climb/descend at most one unit at a time, they may place length 1 scaffolding blocks not part of the desired structure to access higher levels of the workspace. Such scaffolding blocks must be removed before the structure is complete.

Definition 5. *A primitive action $A = (K, B, L)$ specifies a kind of action K, a block B, and a target pose L.*

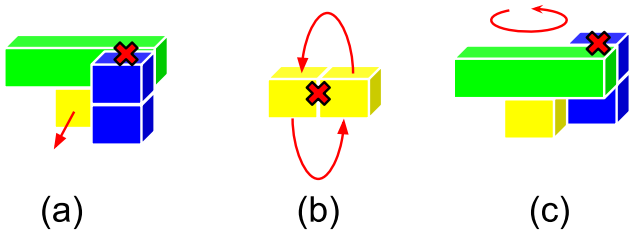


Fig. 4: (a) Robots may not move into a state where they or a carried block intersect other blocks or robots (b) robots cannot exchange locations (c) robots cannot rotate if doing so would intersect other blocks or robots.

Definition 6. *A primitive edge (X, A) consists of a start robot state X and a compatible primitive action A. For WAIT, X must be at the depot. For MOVE, the action's pose must be connected to X. For PICKUP/PLACE, X must be at an access location. For PICKUP, X must not be carrying a block. For PLACE, X must be carrying the block.*

Unlike Cube-MACC, in Cuboid-MACC carried blocks might collide with each other, the world, or other robots (see Fig. 4). Our collision checks are conservative; they may forbid some feasible motions but do not permit any collisions. At both the start and exit states of an edge, the robot and its carried block cannot intersect any other robot or block (a). Additionally, robots or their carried blocks cannot exchange occupied cells during a single timestep (b), and a robot may only rotate if every cell in the axis-aligned square that circumscribes the start and end cells of the carried block is empty (c). The depot location is always collision-free. We now formalize when a robot can occupy a state and execute a primitive action.

Definition 7. *A robot **can stand** at (x, y, z) if $z = 0$ or the cell $(x, y, z - 1)$ is filled.*

Definition 8. *A robot state is **legal** in a world state if it can stand there and neither the robot cell nor the cells occupied by its carried block are filled.*

Definition 9. *A primitive action (K, B, L) is **feasible** in a world state for which it makes sense. MOVE and WAIT are always feasible. PICKUP is feasible if B is at L. PLACE is feasible if B has support at L and the cells it fills are empty.*

Definition 10. *A primitive edge is **executable** in a world state if its primitive action can be executed from its start state. PICKUP, PLACE, and WAIT are always executable. MOVE is executable if the cells swept out by the robot and its block during the movement are all empty.*

Definition 11. *A primitive edge is **valid** in a world state if its start and exit states are legal and its action is both feasible and executable. A sequence of primitive edges is **valid** if every edge is valid.*

To guarantee correctness of any planned execution, it suffices to ensure (i) robots do not violate the robot-robot collision constraints (vertex/edge), and (ii) every executed primitive edge is valid at the time it is executed.

An abstract action thus corresponds to a set of edge sequences. The primitive sequences for ADD are all paths a robot could take from the depot to place the block and return; for REMOVE, the sequences are all paths from the depot to pick-up the block and return.

IV. Hierarchical Planner for Cuboid-MACC

We take a hierarchical approach to Cuboid-MACC as shown in Fig. 2. The high-level searches for the shortest sequence of abstract actions that completes the structure and corresponds to at least one valid concatenation of primitive edge sequences. This is a plan for a single robot to build the structure. Next, we construct an action dependency graph (ADG), a directed acyclic graph whose nodes are the abstract actions in the sequence. A valid primitive sequence to complete an abstract action is exists if all abstract actions corresponding to its ancestors in the ADG are already completed. Finally, we compute multi-agent primitive sequences to complete all the abstract actions by repeatedly selecting all abstract actions corresponding to nodes with no parents in the ADG, solving a multi-agent pathfinding instance, and removing those actions' nodes from the ADG.

A. High-Level: Assembly Action Planning

High-level planning searches a graph where nodes are world states and edges are abstract actions. A* search with edge costs of 1 seeks the shortest sequence of abstract actions building the structure in an initially empty world.

Definition 12. *A pose L is **reachable** in a particular world state with a block B if a valid primitive edge sequence exists from L with B to the depot.*

Definition 13. *ADD or REMOVE block B at pose L is **executable** if at least one access location is reachable both before and after executing the abstract action. If ADD, the access location must be reachable in the original world state while carrying B; if REMOVE, the access location must be reachable in the resulting world state while carrying B.*

At each high-level node, the available edges correspond to the executable abstract actions at that world state (not to primitive edges). The heuristic function estimates the number of remaining ADDs and REMOVEs to build the structure. If n_g unplaced structure blocks with n_p placed scaffolding blocks need n_s additional scaffolding blocks, the exact cost-to-go is: $h^* = (n_g + n_s) + (n_s + n_p)$

n_g and n_p are easy to compute, but the number of ways to construct a scaffolding staircase is extremely large; exactly calculating n_s is challenging [14]. Instead we estimate n_s. Let N be the number of unique heights in the structure where goal block ADDs are required but not executable. We estimate $n_s = N(N+1)/2$. This stems from the pattern that building scaffolding typically requires constructing many towers of decreasing heights (to reach height H, build H-1, H-2, ... 1 height towers). Detailed analysis of this heuristic is beyond the scope of this work, but it is not admissible.

Additionally, the performance of high-level search is dependent on efficient computation of which abstract actions are executable. The naive approach of searching from the boundary to every access location for every abstract action every time a search node is expanded is expensive. Instead, for every world state encountered during the high-level search, we maintain for each block type a set of poses we have proven reachable carrying that block type, and another set of poses we have proven are unreachable. Initially, no poses are unreachable and only the boundary is reachable. When an action pose is queried and it is not in either the reachable or unreachable sets, we conduct breadth first search from the pose until a known reachable or unreachable pose is found. If the reachable set is found, all poses encountered during the search are marked reachable. Conversely, if the unreachable set is found or search terminates without finding anything, all poses encountered are marked unreachable.

B. Interface: Action Dependency Graph Construction

The high-level plans a sequence S of abstract actions corresponding to a valid sequence of primitive edges to construct the target structure. However, many of the abstract actions do not need to occur consecutively. For the ith abstract action in S, we wish to identify a subsequence D of $S[1:i-1]$ of parent abstract actions such that $S[i]$ is executable for the world state produced by executing D. We will then store the relationship between $S[i]$ and the elements of D in the directed Action Dependency Graph (ADG).

For each abstract action $S[i]$, we use A* to find the shortest primitive edge sequence $E[i]$ that accomplishes $S[i]$ and returns to the depot, that is valid in the world state produced by $S[1:i-1]$. Then, for every pair of abstract actions $S[i]$ and $S[j]$, $i < j$, we detect conflicts between the primitive paths $E[i]$ and $E[j]$ (see section IV-C). If a conflict is found, we add an edge from $S[i]$ to $S[j]$.

C. Low-Level: Heterogeneous Task Allocation and Multi-Agent Pathfinding

The interface stage provides a directed acyclic graph whose nodes correspond to abstract actions and whose edges indicate an abstract action must be completed before its child abstract action. At the low-level, we remove abstract actions from this graph, allocate them to robots, and conduct multi-agent pathfinding to obtain primitive edge sequences.

The low-level proceeds in rounds. For each robot, we maintain a record of the earliest robot state at which it is available for a new task. Initially, all robots are available at t=0 and located at the depot.

Within a round, we remove all nodes from the action dependency graph that have no parents. These are the actions that will be completed during this round, and one robot will be involved in the multi-agent pathfinding for each action. For every robot-abstract action pair $\langle R, A \rangle$ we compute the horizontal Manhattan distance between the (x, y) position of R and the nearest access location for A, plus the time at which the R becomes available. This is an underestimate of the time R would complete A. We then solve a min-max job scheduling problem to assign robots to abstract actions, such that the predicted time at which the last action is executed (the "makespan") is minimized. We conduct a bisection search on the makespan; for a given candidate makespan, we find an assignment that achieves this makespan (if it exists) via the Hopcroft-Karp algorithm [18].

Once all abstract actions for the round are assigned ($\langle R_i, A_i \rangle$), we conduct conflict-based search (CBS) [19] between the robots for this round to find primitive edge sequences that complete the assigned action, then return to the depot. The nodes of the CBS internal single-agent searches are robot states. The edges are primitive edges as described in Section III. The conflict-based search algorithm differs from the standard in four ways. First, in every round except the first, universal constraints prevent all robots from interfering with the primitive sequences from previous rounds. This is implemented via a variant of the traditional Edge Constraint that we call a Cell Set constraint.

Definition 14. *A **Cell Set** constraint $(t, \{(x_1, y_1, z_1)...(x_N, y_N, z_N)\})$ specifies a set of cells that neither a robot nor a block may intersect at time t.*

Second, after the first round R_i has edge constraints forbidding every edge that completes A_i before one of its parents. All parents of the A_i were processed in a previous round, so this time is known. Third, the other robots may change the world state. The validity of primitive edges depends on time. At the root node of the CBS tree, we must plan for all robots in the round without knowing when the other robots will change the world. The CBS perspective is to plan individually, assuming that other agents will not interfere, than resolve any conflicts that do arise. Extending this to the mutable world setting, we allow individual robot plans to occupy states and take actions that optimistically will be possible. We introduce the notions of "maybe" states and actions, which can appear in valid single agent plans if other robots complete the correct abstract actions in time. Single agent plans can include maybe states and actions, and which can lead to three new kinds of conflict.

Definition 15. *A **pending action** at time t is an abstract action that may or may not be completed by time t.*

Definition 16. *A robot state (t,L,B) is **maybe legal** in a world state if 1. it can stand at L or the cell below L will be filled by a pending action and 2. for every cell occupied by the robot or its carried block, that cell is either empty or a pending action will empty it.*

Definition 17. *A primitive action is **maybe feasible** if it is feasible. A PICKUP action is maybe feasible if a pending action will place the block to be removed. A PLACE action is maybe feasible if the block would collide with an existing block that a pending action will remove, or pending actions will place enough blocks that the block will have support.*

Definition 18. *A primitive action is **maybe executable** if it is executable. A MOVE action is maybe executable if for each cell swept out by the robot and its block during the motion, the cell is empty or a pending action will empty it.*

Definition 19. *A primitive edge is **maybe valid** if both its start and exit states are maybe legal and its action is both maybe feasible and maybe executable. A sequence of primitive edges is **maybe valid** if every edge is maybe valid.*

During the root internal CBS single agent searches, edges are permitted if they are maybe valid with pending actions consisting of the abstract actions of the other agents for this round. This creates the possibility of a new category of conflict: a primitive edge that is not valid for the world state when the edge is taken. We define three new conflict types, which correspond to states that were not legal, actions that were not feasible, and actions that were not executable. Note that for each conflict, we describe the constraints added to each branch of the conflict tree.

Definition 20. *A **Robot-World** conflict occurs when the exit state $X = (t, L, B)$ of agent i's edge is not legal because of a PICKUP or PLACE action P taken by agent j at time τ. If $t \leq \tau$, the first branch bans agent i from occupying L until $\tau + 1$. The other branch requires agent j to complete P before t. If $t > \tau$, the first branch bans agent i from occupying L after $\tau - 1$. The other branch requires agent j to complete P after t.*

Definition 21. *An **Action-World** conflict occurs when the action A taken by agent i at time t is not feasible because of a PICKUP or PLACE action P taken by agent j at time τ. If $t+1 < \tau$, the first branch bans agent i from any edge with action A at or before τ. The other branch requires agent j to complete P before $t + 1$. If $t + 1 \geq \tau$, the first branch bans agent i from any edge with action A after $\tau - 1$. The other branch requires agent j complete P after $t + 1$.*

Definition 22. *A **Edge-World** conflict occurs when the action A of agent i's edge $E = (X, A)$ is not executable at time t because of a PICKUP or PLACE action P taken by agent j at time τ. If $t + 1 < \tau$, the first branch bans agent i from taking A from the start state of E at any time $\leq \tau$. On the other branch, require agent j to complete P before $t + 1$. If $t + 1 \geq \tau$, the first branch bans agent i from taking A from X at any time $\geq \tau$. The other branch requires agent j complete P after $t + 1$.*

When selecting a conflict to resolve, prioritize Action-World conflicts, then Robot-World conflicts, then Edge-World conflicts before vertex or edge conflicts.

The last variation from standard CBS is in the objective. CBS can minimize the sum of the edge costs of the single agent plans (the "sum-of-costs" formulation) or the maximum single agent plan cost (the "makespan" formulation) [19]. In the multi-agent construction literature (e.g. [12], [27]) it is common to minimize lexicographically (makespan, sum-of-costs). The difficulty in using this objective for CBS is that, when a single agent plans, it should seek smaller edge costs only on paths whose makespan is not the largest amongst all agents. Below the root node, only one agent plans and so the makespan limit is known. Only the root CBS node at which all agents must plan is a challenge. At the root, we first plan with each agent seeking to minimize its individual makespan. Then we replan for each agent, using the makespan limit obtained from the first plan. CBS proceeds as usual from this second set of plans.

D. Correctness and Completeness

We prove that if the pipeline returns a solution it is valid, and that if a single-agent plan to build the structure exists, the pipeline will return a solution.

Lemma 1 (All invalid executions induce a detected conflict)**.** *Consider a set of primitive edge sequences, one per robot, for a low-level round. If a collision occurs or a primitive edge is invalid, there exists a vertex conflict, an edge conflict, or a Robot-World / Action-World / Edge-World conflict.*

Proof. If the collision constraints are violated, there is a vertex or an edge conflict. Suppose some primitive edge is invalid when executed. An edge is invalid iff it is not legal, not feasible, or not executable (Defs. 8-10). By the world conflict definitions (Defs. 20-22), these cases yield a Robot-World, Action-World, and Edge-World conflict respectively. Hence an invalid executed edge induces a world-conflict. □

Lemma 2 (World-conflict branching preserves feasibility)**.** *Consider any CBS node whose constraint set admits a valid joint primitive solution Π. If the node contains a Robot-World, Action-World, or Edge-World conflict, then at least one of the two child nodes produced by resolving that conflict also admits a valid joint primitive solution (in particular, Π satisfies the constraints of at least one child).*

Proof. Fix any valid joint primitive solution Π consistent with the constraints at the current CBS node, and consider the detected world-conflict between agent i and agent j's PICKUP/PLACE P at time τ. In each conflict definition, CBS creates two children: (1) a *ban* branch that forbids the offending state/action/edge of agent i in a specified time range, and (2) an *order* branch that constrains P to occur on the other side of the relevant time boundary (either before

or after). **Robot-World.** If $t \leq \tau$, then either Π has P occur before t (so Π satisfies the order branch), or else P occurs at/after t, in which case Π cannot legally occupy L from t through τ and therefore satisfies the ban branch (ban L until $\tau+1$). If $t > \tau$, the same dichotomy holds with the opposite ordering: either P occurs after t (order branch) or else P occurs at/before t, in which case Π cannot legally occupy L after $\tau-1$ and satisfies the ban branch. **Action-World.** Feasibility is evaluated at the action completion time $t+1$. If $t+1 < \tau$, then either Π schedules P before $t+1$ (order branch) or else P occurs at/after $t+1$, in which case Π cannot execute A at or before τ and satisfies the ban branch. If $t+1 \geq \tau$, either Π schedules P after $t+1$ (order branch) or else P occurs at/before $t+1$, in which case Π cannot execute A after $\tau-1$ and satisfies the ban branch. **Edge-World.** The argument is identical to Action-World, replacing feasibility with executability of A from the start state of $E = (X, A)$ at time t. Thus for each world-conflict, Π satisfies the constraints of at least one child. Hence at least one branch preserves feasibility. □

Theorem 1 (Soundness of modified CBS)**.** *If the low-level search returns primitive plans for a round, then the plans are collision-free and all executed edges are valid in the resulting time-indexed world.*

Proof. CBS returns a solution only if it contains no conflicts. By Lemma 1, any collision or invalid executed edge would imply a vertex conflict, edge conflict, or a world-conflict. Since none exist, the returned plans are collision-free and all executed edges are valid. □

Theorem 2 (Completeness of modified CBS)**.** *If a valid joint primitive solution exists for a round under the given constraints, the modified CBS will find one.*

Proof. For every conflict, at least one branch preserves any valid joint solution ([19] and Lemma 2). Since the constraint tree is explored systematically and single-agent search is complete under constraints, a node consistent with a valid solution will eventually be reached. □

Theorem 3 (Pipeline correctness)**.** *If in a well-formed instance [14] there exists a valid single-agent primitive plan to build the structure, then the pipeline returns a valid multi-agent primitive plan.*

Proof. Assume there exists a valid single-agent primitive plan that constructs the target structure.

High-level completeness. Since a valid single-agent construction exists, there exists a corresponding sequence of executable abstract actions. The high-level A* search explore the abstract state graph and is complete as a graph search, even with an inadmissible heuristic. Therefore, it will eventually find an executable abstract action sequence.

Interface layer. The interface constructs the ADG by inserting dependencies only when conflicts are detected between corresponding single-agent realizations. By construction, actions placed in the same round have no such conflicts. Hence, there exists a sequential single-agent primitive plan that executes all actions in each round.

Low-level completeness. Each round therefore admits at least one valid joint primitive solution (in particular, a sequential single-agent execution). Because the instance is well-formed and CBS is complete for vertex, edge, and world conflicts (Theorem 2), CBS will eventually find a multi-agent plan for each round.

Concatenating the plans for all rounds yields a valid multi-agent primitive plan that constructs the structure. □

V. Results

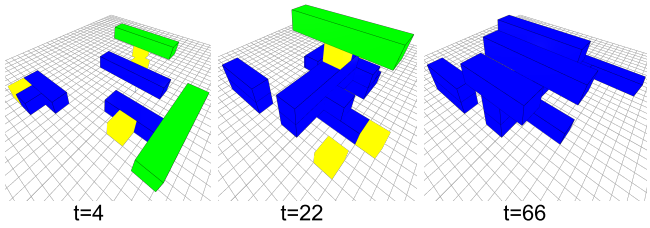


Fig. 5: Images showing the progress of construction of one of the random structures. Robots are in yellow, carried blocks in green, structure blocks in blue.

TABLE I: Walltime in each stage

	High-Level (s)	Interface (s)	Low-Level (s)
Min	0.561	0.0400	1.47
Median	4.94	0.538	30.0
Mean	56.2	0.726	57.0
Max	686	4.97	728

We demonstrate our approach on a test set of 200 randomly generated designs containing 15 blocks stacked up to 4 blocks high in a $10 \times 10 \times 5$ world. In addition to $1 \times 1 \times 1$ blocks, these structures contain $3 \times 1 \times 1$ and $5 \times 1 \times 1$ blocks that are new in the Cuboid-MACC formulation. No prior MACC approach can build them without modifying the algorithm; this paper shows what is required for the hierarchical approach of [14]. Three timesteps from the multi-agent plan obtained for one of these structures are shown in Fig. 5.

Task allocation is handled by the NetworkX 3.6.1 Hopcroft-Karp implementation. Our algorithms are implemented in Python 3.11.14, except for the incremental reachability computations which are implemented in C++ and wrapped by pybind11. We do not leverage multi-threading. Timings are reported for an AMD Ryzen 9 PRO 8945HS with 64GB of RAM. CBS uses bypassing [28]. We allow up to 1000 seconds total walltime for each structure.

Of the 200 test structures, the high-level planner is unable to find an abstract action sequence for 42 of them. In each case, RAM was exhausted during the A* search. The 1000 second limit is reached during the low-level in a further 24 structures. Table I reports summary walltime statistics for the 134 complete successes. The maximum runtime for high- and

low-level search is much higher than the median, suggesting that structures of similar size (15 blocks) vary widely in difficulty. In 101 cases the low-level took the longest; the high-level was slowest in the remaining 33.

VI. Conclusion

In this work, we describe a generalization of the MACC problem to structures composed of heterogeneous cuboid-shaped blocks and demonstrate a hierarchical planning approach to this new problem. We also provide a detailed development of the interaction between the mutability of the world and conflict-based multi-agent pathfinding. The definition of a "maybe valid" edge results from conjunctions and disjunctions of preconditions for the edge to be valid. A more abstract treatment of world mutability and multi-agent conflicts, in terms of action pre and post conditions, may permit extending this approach to more complex settings. We will also investigate further performance improvements; in particular, the hierarchical breakdown of the problem precludes optimality guarantees, so incorporating suboptimal methods throughout the pipeline could provide significant speed ups without sacrificing any theoretical properties. We made use of an inadmissible heuristic for the high-level A* search, but the task allocation and low-level planning both attempt to solve their subproblems to optimality. The propensity of the high-level search to generate extremely large numbers of search nodes and exhaust the available RAM might suggest adopting limited memory search techniques, but the authors suggest that problem-specific heuristics would also be of significant benefit here. Anecdotally, it is not difficult for a human to derive abstract action sequences to build the structures studied in this paper; capturing this ability in heuristic guidance for the search would be fruitful. Finally, the MAPF problems dominate the runtime for most structures; more sophisticated algorithms for multi-agent pathfinding would be beneficial here, such as the use of incremental single agent search in [29].

References

[1] I.-W. Park, D. Catanoso, O. Formoso *et al.*, "Soll-e: A module transport and placement robot for autonomous assembly of discrete lattice structures," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 10 736–10 741.

[2] W. R. Doggett, J. Dorsey, J. Teter *et al.*, "Persistent assets in zero-g and on planetary surfaces: Enabled by modular technology and robotic operations," in *2018 AIAA SPACE and Astronautics Forum and Exposition*, 2018, p. 5305.

[3] H. Thronson, B. M. Peterson, M. Greenhouse *et al.*, "Human space flight and future major space astrophysics missions: servicing and assembly," in *UV/Optical/IR Space Telescopes and Instruments: Innovative Technologies and Concepts VIII*, vol. 10398. SPIE, 2017, pp. 373–388.

[4] S. Scherzinger, J. Weinland, R. Wilbrandt *et al.*, "A walking space robot for on-orbit satellite servicing: The recobot," in *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*, 2022, pp. 2231–2237.

[5] S. Nakanose and K. Nakamura-Messenger, "Gitai usa: Providing safe and affordable means of labor in space," in *ASCEND 2023*, 2023, p. 4744.

[6] S. J. Schuldt, J. A. Jagoda, A. J. Hoisington *et al.*, "A systematic review and analysis of the viability of 3d-printed construction in remote environments," *Automation in Construction*, vol. 125, p. 103642, 2021.

[7] K. H. Petersen, N. Napp, R. Stuart-Smith *et al.*, "A review of collective robotic construction," *Science Robotics*, vol. 4, no. 28, p. eaau8479, 2019.

[8] I. Kostitsyna, J. Gloyd, and K. Cheung, "Multi-agent collective construction of general modular structures," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 12 112–12 118.

[9] K. Petersen, R. Nagpal, and J. Werfel, "Termes: An autonomous robotic system for three-dimensional collective construction," in *Robotics: Science and Systems Conference (RSS)*, 2011.

[10] T. Kumar, S. Jung, and S. Koenig, "A tree-based algorithm for construction robots," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 2014, pp. 481–489, 05 2014.

[11] G. Sartoretti, Y. Wu, W. Paivine *et al.*, "Distributed reinforcement learning for multi-robot decentralized collective construction," in *International Symposium on Distributed Autonomous Robotic Systems*, 2018.

[12] E. Lam, P. J. Stuckey, S. Koenig *et al.*, "Exact approaches to the multi-agent collective construction problem," in *Principles and Practice of Constraint Programming*, H. Simonis, Ed. Cham: Springer International Publishing, 2020, pp. 743–758.

[13] A. K. Srinivasan, S. Singh, G. Gutow *et al.*, "Multi-agent collective construction using 3d decomposition," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 9963–9969.

[14] S. Singh, G. Gutow, A. K. Srinivasan *et al.*, "Hierarchical propositional logic planning for multi-agent collective construction," *Construction Robotics Workshop*, 2023.

[15] A. Grushin and J. A. Reggia, "Automated design of distributed control rules for the self-assembly of prespecified artificial structures," *Robotics and Autonomous Systems*, vol. 56, no. 4, pp. 334–359, 2008.

[16] A. Panangadan and M. G. Dyer, "Construction in a simulated environment using temporal goal sequencing and reinforcement learning," *Adaptive Behavior*, vol. 17, no. 1, pp. 81–104, 2009.

[17] S. R. B. dos Santos, S. N. Givigi, and C. L. Nascimento, "Autonomous construction of structures in a dynamic environment using reinforcement learning," in *2013 IEEE International Systems Conference (SysCon)*, 2013, pp. 452–459.

[18] J. E. Hopcroft and R. M. Karp, "An nˆ5/2 algorithm for maximum matchings in bipartite graphs," *SIAM Journal on computing*, vol. 2, no. 4, pp. 225–231, 1973.

[19] G. Sharon, R. Stern, A. Felner *et al.*, "Conflict-based search for optimal multi-agent pathfinding," in *Artificial Intelligence*, 2012.

[20] J. Chen, J. Li, Y. Huang *et al.*, "Cooperative task and motion planning for multi-arm assembly systems," 2022.

[21] P. Bercher, S. Keen, and S. Biundo-Stephan, "Hybrid planning heuristics based on task decomposition graphs," *Proceedings of the International Symposium on Combinatorial Search*, 2014.

[22] R. Stern, N. R. Sturtevant, A. Felner *et al.*

[23] H. Ma, J. Li, T. K. S. Kumar *et al.*, "Lifelong multi-agent path finding for online pickup and delivery tasks," in *Adaptive Agents and Multi-Agent Systems*, 2017.

[24] H. Choset, K. Lynch, S. Hutchinson *et al.*, *Principles of Robot Motion: Theory, Algorithms, and Implementations*, ser. Intelligent Robotics and Autonomous Agents series. MIT Press, 2005.

[25] I. Kennedy, Monroe, D. Thakur, M. Ani Hsieh *et al.*, "Optimal Paths for Polygonal Robots in SE(2)," *Journal of Mechanisms and Robotics*, vol. 10, no. 2, p. 021005, 02 2018.

[26] P. K. Agarwal, E. Flato, and D. Halperin, "Polygon decomposition for efficient construction of minkowski sums," *Computational Geometry*, vol. 21, no. 1, pp. 39–61, 2002, sixteenth European Workshop on Computational Geometry - EUROCG-2000.

[27] A. Jain, T. Thormählen, and T. a. o. Ritschel, "Exploring shape variations by 3d-model decomposition and part-based recombination," *Computer Graphics Forum*, vol. 31, pp. 631–640, 05 2012.

[28] E. Boyarski, A. Felner, G. Sharon *et al.*, "Don't split, try to work it out: Bypassing conflicts in multi-agent pathfinding," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 25, 2015, pp. 47–51.

[29] E. Boyarski, A. Felner, D. Harabor *et al.*, "Iterative-deepening conflict-based search," in *International Joint Conference on Artificial Intelligence-Pacific Rim International Conference on Artificial Intelligence 2020*. Association for the Advancement of Artificial Intelligence (AAAI), 2020, pp. 4084–4090.